Cookify

El motor que sostiene la promesa: un servicio que **mide el precio en el supermercado** en lugar de estimarlo, y se lo sirve a la app.

22	183	4.934
rutas en el contrato de la interfaz	fuentes de precio verificadas una por una	productos leídos entre las dos cadenas, en una corrida de cada una

DOCUMENTO TÉCNICO DEL BACK END · HITO 2

Proyecto semestral · Arquitectura de Desarrollo (Móvil y Web)

Ernesto Guillén Guerrero
300050336

INGT1003
Escuela de Ingeniería Civil · Universidad Mayor
Segundo semestre 2026

Índice

Introducción 3

1 · Qué cambió respecto del Hito 1 4

1.1 Lo que se mantuvo 4
1.2 Los seis cambios y su motivo 4
1.3 Qué impacto tuvo cada uno 5

2 · Diseño de la interfaz de programación 6

2.1 Los cinco recursos y por qué esos 6
2.2 Códigos de estado y forma del error 7
2.3 Autenticación y criterio de versionado 8

3 · Separación de responsabilidades 9

3.1 Qué archivo es de qué capa 9
3.2 Una petición de lectura, capa por capa 10
3.3 Una recolección, capa por capa 11

4 · Persistencia y credenciales 12

4.1 El modelo de datos 12
4.2 Seis decisiones del esquema 13
4.3 Reproducible, y sin credenciales adentro 14

5 · Decisiones de tecnología 15

5.1 Lenguaje 15
5.2 Framework del servicio 16
5.3 Motor de datos y acceso a datos 16

6 · Monolito o distribuido 18

7 · El recolector y lo que midió 19

7.1 Cómo funciona 19
7.2 El mapeo, que es el trabajo de verdad 20
7.3 Resultados de la medición 21

8 · Cómo se verifica que esto funciona 22

Conclusión 23

Bibliografía 24

Anexo A · Tabla completa de rutas 25

Anexo B · Salida de la verificación 26

INTRODUCCIÓN

El Hito 1 prometió un precio real. Esto es lo que costó cumplirlo.

La propuesta de septiembre decía que Cookify arma una semana de almuerzos con el precio real del supermercado donde se compra. La palabra que había que sostener era «real».

En el Hito 1 ese precio no se medía: salía de 98 valores escritos a mano multiplicados por una constante por cadena. El documento de propuesta ya declaraba que esa constante era una deuda, y la justificaba con una medición propia: comparando los diecisiete productos idénticos presentes el mismo día en Jumbo y en Santa Isabel, la diferencia entre cadenas iba de **–9,1 % a +57,4 % según el producto, no según la tienda**. Si el factor por cadena existiera, las diecisiete diferencias serían el mismo número.

Este hito salda esa deuda. Hay un servicio del lado del servidor que lee el catálogo de las cadenas, normaliza cada producto a la unidad en que el recetario mide sus ingredientes, y responde el precio por una interfaz de programación que la app consume sin saber de dónde salió.

> LO QUE ESTE DOCUMENTO SOSTIENE
>
> Que el servicio funcione es el piso. Lo que sigue explica **por qué está construido así**: por qué Kotlin y no Node, por qué Ktor y no Spring, por qué PostgreSQL y no SQLite, por qué un monolito, y qué costo se aceptó con cada una de esas decisiones.

Lo que hay, en una página

EL SERVICIO

Kotlin 2.3 sobre Ktor 3.2, en cuatro capas separadas de verdad. 22 rutas, cinco recursos del dominio, tres de ellos con las cuatro operaciones completas.

LA PERSISTENCIA

PostgreSQL 18 con cuatro tablas y una vista. El esquema es un script SQL versionado que crea la base desde cero; el servicio no genera tablas.

EL RECOLECTOR

Lee 52 categorías de cada cadena, extrae los productos del catálogo que la página trae embebido, y deja 183 precios medidos: uno por cada fuente activa.

LA VERIFICACIÓN

33 pruebas automatizadas sobre la capa de negocio, 48 verificaciones del contrato de la interfaz, y las 196 fuentes de precio revisadas una por una contra el sitio.

Cómo leer este documento

El capítulo 1 dice qué se movió respecto de lo prometido y por qué: es el capítulo que conviene leer con el documento del Hito 1 al lado. Los capítulos 2, 3 y 4 son el servicio por dentro —contrato, capas, datos—. El 5 y el 6 son las decisiones y su costo. El 7 cuenta la medición y lo que arrojó, que es la parte que hace verdadera la palabra «real». El 8 explica cómo se comprueba todo lo anterior sin creerle a nadie.

CAPÍTULO 1

Qué cambió respecto del Hito 1

Un cambio sin motivo documentado es indistinguible de un error. Estos son los seis que hubo, con lo que los provocó y lo que costaron.

1.1 Lo que se mantuvo

Antes de la lista de cambios conviene decir qué no se tocó, porque es la mayor parte:

— **La promesa.** Sigue siendo una semana de almuerzos que cabe en el presupuesto, con el precio del supermercado donde se compra.

— **El contrato del dominio.** La app define `PrecioRepository` como interfaz y `:core:data` la implementa. El servidor cambia quién responde el precio, no quién lo usa: el motor de planificación no se entera.

— **Categorías y no búsqueda.** La decisión que sostiene todo el servicio. Buscar el nombre del ingrediente daba precios absurdos —medido: «leche» devolvía una mediana de $8,74 por mililitro contra $1,06 real, porque entraba leche condensada y en polvo—. Dentro de la categoría el problema desaparece por construcción.

— **El percentil 25.** Se toma el percentil 25 de los candidatos y no la mediana, porque el planificador asume que quien cocina con presupuesto contado compra la marca barata, no la del medio.

— **El respaldo al catálogo local.** Sin él no se puede armar una semana sin señal.

1.2 Los seis cambios y su motivo

Cambio 1 · El parser de tamaño de envase dejó de ser el camino principal

El Hito 1 planificaba deducir el tamaño del envase desde el nombre del producto —`1 kg`, `900 ml`, `30 un.`— y dividir el precio por él. Era el punto más frágil del diseño y el documento lo reconocía: de las muestras medidas, 34 de 40 nombres resultaban legibles.

Al implementarlo se descubrió que **no hace falta**: la cadena publica en su propio catálogo el campo `ppumListPrice` junto con `ppumMeasurementUnit`, es decir el precio por unidad de medida ya calculado por ella. En las dos cadenas, los 59 productos de la primera página de verduras lo traían completo.

> MOTIVO
>
> El dato que se iba a deducir está publicado. Deducirlo igual sería agregar una fuente de error propia sobre un dato que la cadena ya entrega.

Cambio 2 · La unidad «g» no significa «por gramo»

Verificando el mapeo contra el sitio aparecieron los condimentos costando diez veces lo que debían, sin que nada fallara. El orégano de 50 g cuesta $1.150 —o sea $23 por gramo— y la cadena publica `ppumListPrice: 230` con unidad `g`. La paprika de 100 g cuesta $1.950 —$19,5 por gramo— y publica 195. Cuando la unidad es `g`, el valor viene **por cada diez gramos**.

Se comprobó contra precio y tamaño en los 176 productos de la categoría de condimentos antes de escribir la conversión, y quedó como caso de prueba.

Cambio 3 · Apareció un factor de conversión que el Hito 1 no previó

El diseño original suponía que bastaba con llevar el precio a gramos, mililitros o unidades. No basta, porque **el catálogo y la cadena no siempre cuentan lo mismo**: el recetario mide el ajo en dientes y la cadena vende cabezas; mide la palta en unidades y la cadena la vende por kilo.

La solución es una columna, `unidades_base_por_medida`, que dice cuántas unidades del catálogo entrega una unidad canónica del producto. El precio sale de una división, y el valor queda documentado y auditable en vez de escondido en el código:

```
cebolla   producto en $/g    1 g      = 1 g de catálogo          → 1
ajo       producto en $/un   1 cabeza = 10 dientes de catálogo   → 10
palta     producto en $/g    1 g      = 1/200 de una palta       → 0,005
```

Cambio 4 · El precio pasó de ser un valor a ser una historia

El Hito 1 describía una tabla `precios_cache` en el teléfono con un valor por par, que se sobrescribía en cada refresco. En el servidor cada medición se **inserta**: así se puede responder cuánto costaba la cebolla hace tres semanas y auditar un precio raro contra la corrida que lo produjo. El vigente sale de una vista.

Cambio 5 · «Medido» o «estimado» dejó de ser una etiqueta de pantalla

El Hito 1 anotaba como pendiente «marcar las 4 cadenas sin datos reales como precio estimado» en la interfaz. Se resolvió un nivel más abajo: `origen` es una columna de la tabla y viaja en cada respuesta. La app no tiene que acordarse de nada.

Cambio 6 · Trece fuentes quedaron inactivas a propósito

El Hito 1 suponía cobertura completa de los 98 ingredientes en las dos cadenas. No la hay. El tofu y la soya texturizada no se venden en el catálogo en línea de ninguna de las dos, y Santa Isabel no vende en línea brócoli, choclo fresco, poroto verde, cilantro, perejil, camote, pavo molido, pasta de tomate ni leche de coco.

Motivo: apuntarlas a una categoría parecida habría dado un número. Un mapeo malo **no falla**: cobra mal y nadie se entera. Se prefirió declararlas inactivas y dejar su precio en `ESTIMADO`, que es verdad.

1.3 Qué impacto tuvo cada uno

CAMBIO	IMPACTO EN EL ALCANCE	IMPACTO EN EL RIESGO
1 · Precio por unidad publicado	Sale del alcance el parser de nombres y sus pruebas	Baja: se elimina la pieza más frágil del diseño
2 · La unidad «g» es por 10 g	Una línea de conversión y un caso de prueba	Baja mucho: era un error silencioso de factor diez
3 · Factor de conversión	Una columna más en `fuente_precio`	Baja: la conversión queda visible y auditable
4 · Historia de precios	Una vista y un parámetro más	Baja: un precio raro se puede rastrear
5 · `origen` en los datos	Una columna; sale un pendiente de interfaz del Hito 3	Baja: la app no puede prometer de más por descuido
6 · Fuentes inactivas	**13 de 196 pares quedan sin medición**	**Sube el honesto: se reconoce lo que no se cubre**

Ninguno obligó a tocar el motor de planificación: consecuencia de que el dominio defina `PrecioRepository` como interfaz.

CAPÍTULO 2

Diseño de la interfaz de programación

El contrato se cerró antes de escribir la primera línea del servicio y está completo en `docs/api-cookify.md`. Lo que sigue es por qué quedó así.

2.1 Los cinco recursos y por qué esos

La interfaz está orientada a recursos del dominio, no a acciones. La prueba de que un recurso está bien elegido es que las cuatro operaciones tengan sentido sobre él sin forzar el lenguaje.

RECURSO	QUÉ ES	OPERACIONES	POR QUÉ ES UN RECURSO
`supermercados`	las seis cadenas que la app ofrece	leer	Catálogo cerrado: agregar una cadena implica construirle un recolector, no insertar una fila
`ingredientes`	los 98 del recetario	crear · leer · actualizar · borrar	El catálogo lo mantiene quien desarrolla, y era un requisito desde el Hito 1
`fuentes`	de dónde se lee el precio de cada ingrediente	crear · leer · actualizar · borrar	Es el mapeo que sostiene el servicio y el que más cambia cuando la cadena mueve su árbol de categorías
`precios`	la medición, con su historia	crear · leer · actualizar · borrar	Es lo que la app consume. Se puede cargar a mano para una cadena sin recolector
`recolecciones`	una corrida del recolector	crear · leer · borrar	Ver abajo

Por qué `recolecciones` es un recurso y no una acción

La tentación era `POST /recolectar`. Se descartó porque una corrida **tiene estado y dura**: seis minutos contra Jumbo. Si fuera una acción, quien la dispara no tendría cómo preguntar cómo va, qué leyó ni por qué falló.

Como recurso, la corrida tiene identidad: se crea con `POST`, que responde **202 Accepted** y la cabecera `Location`; se sigue con `GET` sobre esa dirección; y queda en el historial con sus métricas y su detalle. El 202 no es un adorno: dice que la petición se aceptó y que el resultado no está listo, que es exactamente lo que pasa.

Dos lecturas distintas sobre `precios`

`GET /api/v1/precios?supermercado=JUMBO` devuelve el precio **vigente** de cada ingrediente, que es lo que la app guarda en su caché. Con `historia=true` devuelve todas las mediciones, paginadas, que es lo que se usa para auditar. Son dos preguntas distintas con dos formas de respuesta distintas, y el parámetro lo dice.

La respuesta vigente lleva `actualizadoEn` al nivel del lote y no solo en cada precio: la app necesita una sola fecha para decidir si su caché de siete días venció.

2.2 Códigos de estado y forma del error

La mitad del trabajo de un servicio es responder bien cuando algo falla. La regla que se hizo cumplir en el diseño es una sola: **ningún error viaja con código 200**. Un error con código de éxito obliga a quien consume el servicio a leer el cuerpo para saber si algo salió bien, y rompe a cualquier cliente que confíe en el estado.

CÓDIGO	CODIGO	CUÁNDO SE DEVUELVE
200	—	lectura, o actualización que devuelve el recurso completo
201	—	creación, con `Location` apuntando al recurso nuevo
202	—	la recolección arrancó; el resultado no está listo
204	—	borrado. Sin cuerpo: el estado ya lo dice todo
400	`VALIDACION`	el cuerpo o un parámetro no cumplen el contrato
401	`NO_AUTORIZADO`	falta la clave o no corresponde
404	`NO_ENCONTRADO`	el recurso pedido no existe
409	`CONFLICTO`	ya existe un recurso con esa identidad
503	`FUENTE_NO_DISPONIBLE`	el sitio de la cadena no respondió
500	`ERROR_INTERNO`	falla no prevista, con su referencia en el registro

El cuerpo del error es siempre el mismo objeto

```
{
  "error": {
    "codigo": "VALIDACION",
    "mensaje": "El cuerpo de la petición no es válido",
    "detalles": [
      { "campo": "precioBaseClp", "problema": "debe ser mayor que 0" },
      { "campo": "unidad",        "problema": "debe ser uno de: g, ml, un" }
    ]
  }
}
```

`detalles` acumula los problemas en lugar de fallar en el primero: quien manda un cuerpo con tres campos malos preferiría enterarse de los tres en una respuesta antes que descubrirlos de a uno en tres intentos.

Tres decisiones que se discutieron

— **POST repetido responde 409, no 200.** Devolver el recurso que ya existía le oculta a quien lo mandó que no lo creó él.

— **Una fuente con un ingrediente inexistente responde 404, no 400.** El cuerpo está bien formado; lo que no existe es aquello a lo que apunta.

— **Recolectar en una cadena sin recolector responde 409.** La petición es válida y la cadena existe: lo que no permite la operación es el estado del sistema.

Cómo se hace cumplir: la traducción de excepción a código vive en un solo lugar, con el plugin `StatusPages`. Las capas de negocio y de datos no mencionan códigos HTTP, y una excepción que se escape sin mapear sale como 500, nunca como éxito.

2.3 Autenticación y criterio de versionado

Quién puede escribir

La lectura es abierta: los precios de un supermercado no son un secreto y la app los pide sin credenciales. Toda escritura —`POST`, `PUT`, `DELETE`— exige la cabecera `X-API-Key`.

La verificación es un plugin que mira cada llamada, no una línea dentro de cada manejador. La diferencia es concreta: con el plugin, una ruta de escritura nueva queda protegida por omisión; con la verificación repartida, queda protegida solo si alguien se acuerda.

> **DOS DETALLES QUE NO SON ADORNO**
>
> La comparación de la clave es de **tiempo constante** (`MessageDigest.isEqual`): el operador == sale en el primer carácter distinto, y esa diferencia de tiempo se puede medir desde afuera para adivinar la clave carácter por carácter. Y se responde **401 tanto si falta como si no corresponde**: distinguir los dos casos le confirma a quien prueba claves que el formato era el correcto.

Criterio de versionado

La versión va en la ruta —`/api/v1`— y no en una cabecera, porque así una petición es legible y reproducible desde la barra del navegador y desde `curl`, que es donde se demuestra y donde se depura.

CAMBIO	¿SUBE LA VERSIÓN?
Agregar un recurso	no
Agregar un campo opcional a una respuesta	no
Agregar un parámetro opcional de consulta	no
Quitar o renombrar un campo	**`/api/v2`**
Cambiar el tipo o el significado de un campo	**`/api/v2`**
Cambiar un código de estado ya documentado	**`/api/v2`**

Que agregar campos no suba la versión no es gratis: exige que el cliente los ignore. El servicio hace lo mismo al recibir (`ignoreUnknownKeys`), de modo que la regla valga en las dos direcciones.

Cuando aparezca `v2`, `v1` se mantiene un ciclo completo de la app en las tiendas antes de retirarse. El cliente que ya está instalado en un teléfono no se puede actualizar por decreto, y ese es justamente el motivo por el que una app móvil necesita versionar su interfaz y una página web puede darse el lujo de no hacerlo.

`/salud` vive fuera del contrato

Responde 200 con la versión y el estado de la conexión a la base, o 503 si la base no responde. Está fuera de `/api/v1` a propósito: no es parte del contrato de datos sino del despliegue, y por eso tampoco cambia cuando cambia la versión de la interfaz. Es lo que usa `docker compose` para saber si el contenedor está sano.

CAPÍTULO 3

Separación de responsabilidades

Tres carpetas con toda la lógica dentro del controlador no es arquitectura en capas. La regla se escribió antes del código y se puede verificar abriendo cualquier archivo.

3.1 Qué archivo es de qué capa

```
server/src/main/kotlin/com/app/cookify/server/
├── Aplicacion.kt            arranque · el grafo de dependencias completo
├── config/
│   ├── Configuracion.kt     el entorno, validado al arrancar
│   ├── Errores.kt           los errores que el servicio sabe nombrar
│   └── Plugins.kt           serialización, registro, errores, clave de la API
├── ruteo/
│   └── Ruteo.kt             presentación · parámetros, cuerpos, códigos, Location
├── negocio/
│   ├── Servicios.kt         negocio · las reglas y las validaciones
│   ├── RecolectorServicio.kt  la corrida contra el sitio de la cadena
│   └── Normalizador.kt      de lo que publica la cadena al precio del recetario
├── datos/
│   ├── FuenteDeDatos.kt     datos · pozo de conexiones y transacciones
│   ├── Daos.kt              el único lugar con SQL
│   └── externo/
│       ├── ClienteCadena.kt   el único lugar que habla con jumbo.cl
│       └── ParserCatalogo.kt  el catálogo embebido en la página
├── dto/Dtos.kt              el contrato HTTP, separado del dominio
└── modelo/Modelos.kt        el dominio: Kotlin puro, sin anotaciones
```

La regla, y cómo se comprueba

CAPA	DE QUÉ RESPONDE	DE QUÉ NO RESPONDE
Presentación `ruteo/`	Leer parámetros, deserializar el cuerpo, elegir el código de éxito, poner `Location`	No valida reglas de negocio, no arma consultas, no toca la base
Negocio `negocio/`	Decidir si una operación vale y qué significa; validar; normalizar	No escribe SQL ni menciona un código de estado HTTP
Datos `datos/`	Guardar, leer y hablar con sistemas de afuera	No decide reglas; traduce choques de la base a excepciones que el negocio entiende
Dominio `modelo/`	Nombrar las cosas: Ingrediente, Fuente, Precio, Recolección	No sabe de HTTP ni de serialización: no tiene una sola anotación

POR QUÉ HAY UN PAQUETE DTO ADEMÁS DE MODELO

Porque son dos contratos distintos con dos ritmos distintos. Hoy casi coinciden y la traducción es trivial; el valor está en que exista el lugar donde hacerla cuando dejen de coincidir. Sin él, renombrar una columna rompería la app, y cambiar la forma de una respuesta obligaría a tocar el SQL.

Que cada manejador de ruta quepa en cuatro líneas no es una casualidad de estilo: es la señal de que no le corresponde hacer más. Un manejador que crece es un manejador que se está quedando con trabajo de otra capa.

3.2 Una petición de lectura, capa por capa

El recorrido completo de `GET /api/v1/precios?supermercado=JUMBO`, que es la petición que la app hace una vez por semana.

1. PRESENTACIÓN · RUTEO.KT
   Netty entrega la llamada. El manejador lee `supermercado`, `origen` e `historia` de la consulta, y con `historia` en falso llama a `precios.vigentes(...)`. No valida nada: no le corresponde.
2. NEGOCIO · PRECIOSERVICIO
   Valida que venga el supermercado —el precio vigente es siempre de una cadena, y sin ella la pregunta no tiene respuesta— y traduce `origen` al enumerado del dominio. Si algo falla lanza `ErrorDelServicio.Validacion` y no vuelve.
3. DATOS · FUENTEDEDATOS
   Abre una transacción. Toma una conexión del pozo de HikariCP y salta a `Dispatchers.IO`: JDBC bloquea el hilo, y bloquear un hilo de Netty deja de atender peticiones que no tienen nada que ver con esta.
4. DATOS · SUPERMERCADODAO
   Comprueba que la cadena exista. Si no, el negocio lanza `NoEncontrado` y la transacción se revierte sin haber escrito nada.
5. DATOS · PRECIODAO
   Consulta la vista `precio_vigente`, que resuelve «la última medición de cada par» con `DISTINCT ON` dentro de la base. Hacerlo en el servicio obligaría a traer la historia completa por la red para descartar casi todo.
6. DATOS → DOMINIO
   Cada fila se mapea a un `Precio` del modelo. A partir de aquí nadie vuelve a tocar un `ResultSet`.
7. PRESENTACIÓN · DTOS.KT
   El manejador convierte la lista a `PreciosVigentesDto`, calcula `actualizadoEn` como la medición más nueva del lote, y responde 200. Las fechas salen en ISO 8601 con desplazamiento explícito.

Si en el paso 2 el supermercado viene vacío, la respuesta es un 400 con el campo nombrado, y los pasos 3 a 7 no ocurren. Si en el paso 4 la cadena no existe, es un 404. En los dos casos la traducción de la excepción al código la hace `StatusPages`, no el manejador.

> **LO QUE ESTO HABILITA**
>
> La app Android llama a este mismo recorrido a través de su `PrecioRepository`, que el dominio del cliente define como interfaz. El motor de planificación pide el precio a esa interfaz y no sabe si lo responde el JSON empaquetado o el servicio. Cambió quién responde el precio, no quién lo usa.

3.3 Una recolección, capa por capa

El recorrido de `POST /api/v1/recolecciones` es más interesante porque cruza hacia afuera y porque no termina cuando termina la petición.

1. PRESENTACIÓN
   El plugin de la clave verifica `X-API-Key` antes que nada; sin ella la llamada ni siquiera llega al manejador. El manejador deserializa el cuerpo y llama a `recolector.iniciar(...)`.

2. NEGOCIO · RECOLECTORSERVICIO
   En **una** transacción: comprueba que la cadena exista y tenga recolector, que no haya otra corrida `EN_CURSO` —dos simultáneas escribirían la misma fila—, carga las fuentes activas y las unidades del catálogo, y abre la fila de la recolección.

3. PRESENTACIÓN
   Responde **202** con `Location`. La petición termina aquí, a los pocos milisegundos. La corrida sigue en el alcance de la aplicación, no en el de la petición.

4. NEGOCIO
   Agrupa las fuentes **por categoría** y no por ingrediente: 96 ingredientes caben en 52 categorías, y pedir una página por ingrediente sería pedir la misma página tres veces para la cebolla, el tomate y la papa.

5. DATOS · CLIENTECADENA
   Por cada categoría pide sus páginas con una pausa entre peticiones. La pausa no es un detalle de rendimiento sino de cortesía: sin ella Santa Isabel empieza a responder 502. Un 404 en una página posterior a la primera es el fin del recorrido, no una falla.

6. DATOS · PARSERCATALOGO
   Extrae los productos del catálogo embebido contando llaves, no con una expresión regular: los objetos vienen anidados y una regex cierra el objeto en el lugar equivocado.

7. NEGOCIO · NORMALIZADOR
   Por cada fuente filtra por términos y exclusiones, lleva el precio a su unidad canónica, descarta lo que viene en otra medida que la esperada —convertirlo sería una invención—, divide por el factor de la fuente y toma el percentil 25.

8. DATOS
   Todo lo medido se escribe en **una sola transacción** junto con el cierre de la recolección. Media cadena medida y media vieja sería imposible de distinguir después.

> **DOS COSAS QUE SE ARREGLARON CONSTRUYENDO ESTO**
>
> El precio se guardaba con la unidad del **producto** en lugar de la del catálogo, y la albahaca aparecía como «$1.320 el gramo» cuando son $1.320 el atado. Y una corrida que quedaba a medias por un reinicio del servicio dejaba la fila `EN_CURSO` para siempre, bloqueando toda recolección futura de esa cadena; ahora el servicio cierra esas corridas huérfanas al arrancar y dice por qué.

CAPÍTULO 4

Persistencia y credenciales

4.1 El modelo de datos



Cuatro tablas y una vista. El esquema completo, con sus comentarios y todas sus restricciones, está en `db/001_esquema.sql`.

TABLA	QUÉ GUARDA	FILAS HOY
`supermercado`	las cadenas y si tienen recolector	6
`ingrediente`	el catálogo del recetario, con su precio de referencia	98
`fuente_precio`	el mapeo ingrediente → categoría de la cadena	196
`recoleccion`	una fila por corrida del recolector	—
`precio`	cada medición; es historia, no estado	588 +
`precio_vigente`	vista: la última medición de cada par	—

4.2 Seis decisiones del esquema

1 · El precio es historia, no estado

Cada medición se inserta y ninguna se sobrescribe. Cuesta espacio —98 ingredientes por cadena por corrida— y lo compra barato: se puede responder cuánto costaba la cebolla hace tres semanas, y un precio raro se puede auditar contra la corrida que lo produjo, porque `precio.recoleccion_id` apunta a ella.

2 · La vista `precio_vigente` resuelve la lectura en la base

`DISTINCT ON (supermercado, ingrediente) ... ORDER BY medido_en DESC` es una extensión de PostgreSQL que devuelve la primera fila de cada grupo. La alternativa portable —una subconsulta con `MAX(medido_en)` y una reunión sobre sí misma— se lee peor y hace más trabajo. Se asume la dependencia del motor a cambio de una consulta de siete líneas.

3 · Las restricciones están en la base, no solo en el código

`precio_clp_por_unidad > 0`, `unidad in ('g','ml','un')`, `id ~ '^[a-z0-9_]{2,48}$'`. El servicio valida lo mismo antes de insertar, y eso no es redundancia: la validación del servicio da un mensaje útil, y la de la base garantiza que el dato no entre aunque alguien escriba con `psql` o aunque mañana haya un segundo servicio.

4 · Un par cadena-ingrediente tiene a lo más una fuente

Es un índice único. Con dos fuentes para el mismo par, el precio dependería de cuál se leyó primero, y la respuesta cambiaría entre corridas sin que nada fallara.

5 · `timestamptz` y no `timestamp`

El precio de un supermercado chileno se mide en hora de Chile. Guardar el instante sin desplazamiento hace que la misma fila signifique cosas distintas según dónde corra el servidor, y el horario de verano convierte eso en una hora de diferencia dos veces al año.

6 · `numeric` y no `float` para el dinero

`numeric(10,4)` con cuatro decimales. No es entero porque en gramos los precios son fracciones de peso —la cebolla vale $1,975 el gramo— y redondear a peso acumula un error grande al escalar a cuatro porciones por siete días. Y no es `double precision` porque el dinero no se representa en binario sin error.

> **UNA RESTRICCIÓN QUE VALE LA PENA MIRAR**
>
> `recoleccion` tiene un `CHECK` que exige que una corrida `EN_CURSO` no tenga fecha de término y que una terminada sí la tenga. Es el tipo de incoherencia que el código deja pasar en el segundo mes del proyecto.

4.3 Reproducible, y sin credenciales adentro

La base se crea desde cero con un script

`db/001_esquema.sql` es la **única** fuente de verdad del esquema. El servicio no crea tablas al arrancar: si las creara habría dos verdades sobre la forma de la base y se separarían en la primera edición.

`db/002_semilla.sql` se **genera** desde el catálogo de la app y desde el mapeo de fuentes, con `tools/generar_semilla.py`, por la misma razón: copiar a mano los 98 ingredientes al SQL crearía dos verdades más.

```
$ cp .env.example .env            # y completar tres valores
$ docker compose up -d db         # aplica 001 y 002 solo, al crearse el volumen
$ ./gradlew -p server run

$ curl -s http://localhost:8090/salud
{"estado":"arriba","version":"1.0.0","baseDeDatos":"responde"}
```

El contenedor de PostgreSQL ejecuta en orden todos los `.sql` de `db/` la primera vez que arranca. Para partir de cero: `docker compose down -v && docker compose up -d db`.

Las credenciales están fuera del repositorio

QUÉ	DÓNDE VIVE	¿SE VERSIONA?
Valores reales	`.env` en el equipo	no · está en `.gitignore`
Nombres y ejemplos	`.env.example` en la raíz	sí
Lectura al arrancar	`config/Configuracion.kt`	sí

El servicio **no arranca** si falta una variable obligatoria. Arrancar con la clave vacía dejaría la interfaz abierta a escritura y nadie se daría cuenta hasta que alguien escribiera. Fallar en el arranque es ruidoso y barato; fallar en producción no es ninguna de las dos cosas.

> **TRES CUIDADOS MENOS OBVIOS**
>
> El mensaje de error de una clave corta informa **el largo y nunca el valor**: un mensaje de arranque termina en un registro y un registro se comparte. El registro de llamadas anota método, ruta y estado, nunca el cuerpo ni las cabeceras, que es donde viaja la clave. Y la colección de Postman deja `apiKey` vacía a propósito: el archivo está en un repositorio público.

Si alguna vez se subiera una credencial, borrarla no alcanza: el historial de git la conserva. Hay que rotarla.

CAPÍTULO 5

Decisiones de tecnología

Cada decisión con su alternativa comparada, los criterios con que se compararon y el costo que se acepta. Una elección sin costo declarado es una elección que no se pensó.

5.1 Lenguaje · Kotlin

CRITERIO	KOTLIN / JVM	TYPESCRIPT / NODE	PYTHON
Coherencia con el cliente	**el mismo lenguaje que la app**	otro lenguaje	otro lenguaje
Tipos en compilación	sí, nulabilidad incluida	sí, borrados en ejecución	opcionales
Concurrencia del recolector	corrutinas, ya conocidas	bucle de eventos	asyncio
Curva para el equipo	nula: ya se escribe	media	baja
Memoria en reposo	~200 MB	~60 MB	~40 MB
Arranque en frío	1 a 2 s	< 0,2 s	< 0,3 s

Se eligió Kotlin. El argumento que pesó no es el rendimiento: es que el equipo ya escribe Kotlin todos los días en la app, y que el modelo del dominio —`Unidad`, `Pasillo`, la normalización de precios— es el mismo concepto en los dos lados. Un lenguaje distinto en el servidor obligaría a mantener dos vocabularios para una misma idea, con una persona. La nulabilidad en el tipo importa más de lo que parece en un servicio cuyo trabajo es interpretar JSON ajeno donde la mitad de los campos pueden faltar.

> **COSTO QUE SE ACEPTA**
>
> La imagen de producción pesa **323 MB** medidos, varias veces lo que ocuparía una equivalente en Node, y el arranque en frío es un orden de magnitud más lento. Para un servicio que corre continuo y se despliega pocas veces por semana, ninguna de las dos cosas se nota. Si se desplegara en funciones que arrancan por petición, esta decisión sería la equivocada.

5.2 Framework del servicio · Ktor

CRITERIO	KTOR 3.2	SPRING BOOT 3
Capas	las impone el equipo	por convención: Controller, Service, Repository
Arranque	~1 s	3 a 6 s
Cuánto hay que poder explicar	lo que está escrito	lo escrito más la configuración automática
Cliente HTTP para el recolector	incluido, con corrutinas	`WebClient`, aparte
Soporte y comunidad	JetBrains, menor	enorme
Respuestas listas a preguntas comunes	pocas	muchísimas

Se eligió Ktor. El criterio que desempató fue el de la defensa individual: en este hito cualquiera del equipo tiene que poder explicar una parte del código y por qué está escrita así. En Ktor el grafo de dependencias completo se lee en `Aplicacion.kt`, de arriba abajo, sin anotaciones que construyan objetos en otro lado. Con Spring habría que poder explicar también lo que la configuración automática hizo sin que nadie lo escribiera.

> **COSTO QUE SE ACEPTA**
>
> Ktor **no impone capas**. Nada impide escribir una consulta SQL dentro de un manejador de ruta, que es exactamente el error que el enunciado llama «capas que son solo carpetas». Spring lo habría hecho más difícil por convención. Se compensa con la regla escrita, con la estructura de carpetas y con que ningún archivo de `ruteo/` importe nada de `datos/`, pero es disciplina y no garantía: hay que sostenerla en cada revisión.

5.3 Motor de datos · PostgreSQL

CRITERIO	POSTGRESQL 18	MYSQL 8	SQLITE
Arreglos de texto nativos	**sí**	no	no
`DISTINCT ON`	**sí**	no	no
Instante con zona	`timestamptz`	parcial	texto
`CHECK` que se cumple	sí	sí (desde 8.0.16)	sí, con tipos débiles
Operación	un contenedor	un contenedor	un archivo

Se eligió PostgreSQL. Dos funciones concretas del modelo lo pedían. Los `text[]` de `terminos` y `excluir` evitan una tabla puente con dos filas por fuente para guardar lo que conceptualmente es una lista corta. Y `DISTINCT ON` resuelve `precio_vigente` en siete líneas. SQLite quedó descartado además por una razón del enunciado: se pide persistencia en un motor de datos, y un archivo junto al ejecutable no se parece a lo que va a haber en el Hito 3.

Costo: PostgreSQL es un servicio aparte que hay que levantar, y eso convierte a Docker en un requisito para desarrollar. Con SQLite no habría nada que levantar.

Acceso a datos · JDBC con HikariCP, sin ORM

CRITERIO	JDBC + HIKARICP	EXPOSED	HIBERNATE / JPA
Dónde vive el esquema	**en SQL versionado**	en código Kotlin	en anotaciones
La consulta que corre	la que está escrita	generada	generada
Código repetitivo	**mucho**	poco	casi nada
Qué hay que poder explicar	SQL	SQL y el DSL	SQL, JPA y el caché de sesión

Se eligió JDBC. El esquema ya es un script SQL versionado —requisito del hito— así que un ORM que también describiera el esquema crearía la segunda verdad que se quería evitar. Además, usar `DISTINCT ON` y `text[]` a través de un ORM significa pelear con la abstracción justo donde se eligió el motor por esas dos funciones.

> **COSTO QUE SE ACEPTA, Y ES EL MÁS CARO DE ESTE CAPÍTULO**
>
> Mapear cada fila a mano es código repetitivo que un ORM daría gratis, y cada columna nueva obliga a tocar tres lugares. Se acepta porque el servicio tiene cuatro tablas, no cuarenta. Con veinte tablas esta decisión se revisaría.

Qué se descartó y por qué

— **GraphQL en lugar de REST.** Tiene sentido cuando hay muchos clientes con necesidades distintas. Aquí hay uno, y pide una sola cosa: los precios vigentes de una cadena. REST con recursos se explica y se demuestra con `curl`.

— **Una interfaz pública de los supermercados.** No existe. En el Hito 1 se probaron y descartaron `catalog_system` de VTEX y `intelligent-search`: las dos responden 404 contra el sitio actual de Jumbo.

— **Un navegador sin interfaz para leer las páginas.** Habría resuelto también aCuenta, que renderiza en el cliente. Cuesta cientos de megas de imagen y segundos por página; la página renderizada en el servidor alcanza para las dos cadenas que importan.

— **Flyway o Liquibase para las migraciones.** Para cuatro tablas y un script, el contenedor de PostgreSQL ejecutando `db/*.sql` en orden alcanza. Se revisaría en cuanto haya que migrar una base con datos en producción.

CAPÍTULO 6

Monolito o distribuido

Se eligió un **monolito modular**: un proceso, una base, cuatro capas separadas por dentro. No por comodidad, y la decisión tiene condiciones de revisión escritas.

Los cuatro argumentos, con números

Carga esperada

La app consulta precios una vez cada siete días por usuario, porque guarda una caché local con vencimiento. Con mil usuarios activos son mil lecturas semanales: del orden de **0,002 peticiones por segundo**. El trabajo pesado no son las lecturas sino el recolector: dos corridas diarias de unas 150 peticiones salientes. Un proceso basta.

Tamaño del equipo

Un equipo chico. Dividir multiplica los despliegues, los contratos entre servicios y los modos de falla. Esa multiplicación se paga con personas que operen el sistema, y no las hay.

Costo de operación

Un contenedor de servicio y uno de base caben en el servidor más barato. Un conjunto de microservicios exigiría además orquestación, descubrimiento y trazas distribuidas: tres piezas de infraestructura que nadie del equipo opera hoy.

Complejidad de despliegue

`docker compose up -d` contra un plano de control que hay que mantener. En el Hito 3 el proyecto tiene que levantar en el equipo del evaluador siguiendo un archivo de instrucciones; cada servicio extra es una forma más de que eso falle delante de él.

> **LO QUE SÍ SE TOMÓ DEL DISEÑO DISTRIBUIDO**
>
> El límite ya está dibujado por dentro. `CatalogoDeCadena` es una interfaz, y el recolector solo depende de ella y de los DAO. Extraerlo como servicio aparte sería reemplazar una implementación por un cliente HTTP: no habría que tocar ni el ruteo ni el resto del negocio. Un monolito con costuras en los lugares correctos es más barato de dividir después que un conjunto de servicios de partir mal.

En qué escenario se revisaría

SI PASARA ESTO	SE HARÍA ESTO
La recolección crece a seis cadenas y a todo el árbol de categorías: del orden de mil peticiones y dos horas por corrida, compitiendo por el pozo de conexiones con las lecturas de la app	Extraer **solo el recolector** a un proceso aparte con una cola de trabajos, dejando la interfaz de lectura donde está
La app pide precios en tiempo real en vez de con caché semanal, y las lecturas pasan a dominar	Poner un caché de lectura delante de la interfaz antes de dividir nada
El equipo crece y dos grupos se estorban en el mismo repositorio	Ahí sí dividir, siguiendo las costuras que ya existen

Lo que no es motivo para dividir: que suene moderno.

CAPÍTULO 7

El recolector y lo que midió

Esta es la parte que hace verdadera la palabra «real» de la promesa del Hito 1.

7.1 Cómo funciona

No hay interfaz pública de precios en ninguna de las cadenas chilenas. Lo que sí hay es la página de categoría renderizada en el servidor, que trae el catálogo embebido en el payload del framework. De ahí se leen, por producto, el nombre, el precio de lista, el precio por unidad de medida y la unidad de esa medida.

Se lee `ppumListPrice` y no `ppumPrice` a propósito: una promoción horneada en el catálogo hace que el presupuesto de la semana se vea optimista y que cambie solo entre dos consultas, sin que el usuario haya hecho nada.

Las dos cadenas que se pueden medir, y las cuatro que no

CADENA	ESTADO	POR QUÉ
Jumbo	✅ recolector	catálogo renderizado en el servidor, 231 categorías de comida
Santa Isabel	✅ recolector	misma plataforma: 214 de sus 226 categorías son idénticas
aCuenta	⚠️ fuera de alcance	el HTML llega, pero los productos se renderizan en el cliente
Unimarc · Líder · Tottus	❌ fuera de alcance	403 del cortafuegos, 302 y 526 respectivamente

Las cuatro restantes siguen con el multiplicador del Hito 1 y su precio queda marcado `ESTIMADO` en la columna `origen`, de modo que la app lo pueda decir.

Una petición por categoría, no una por ingrediente

Las fuentes se agrupan antes de salir a la red: 96 ingredientes activos de Jumbo caben en 52 categorías. Pedir una página por ingrediente sería pedir la misma página de verduras tres veces, para la cebolla, el tomate y la papa.

> **POR QUÉ ESTO VIVE EN UN SERVIDOR Y NO EN EL TELÉFONO**
>
> Medido: una corrida completa son 52 categorías, unas 150 peticiones, más de tres mil productos y seis minutos. La pantalla de armado de la app dura 3,5 segundos. Además, una descarga aquí sirve a todos los clientes; mil teléfonos haciendo lo mismo contra el sitio de la cadena es otra cosa. Y cuando la cadena cambie su sitio, se arregla un servicio en una tarde, no una app instalada que tiene que pasar por la tienda.

7.2 El mapeo, que es el trabajo de verdad

Una fuente dice de qué categoría del sitio se lee el precio de un ingrediente, con qué términos se lo reconoce dentro de ella, qué variedades se excluyen, en qué unidad publica la cadena y por cuánto hay que dividir para llegar a la unidad del recetario.

```
{
  "ingredienteId": "cebolla",
  "categoria": "/frutas-y-verduras/verduras/verdura",
  "terminos": ["cebolla"],
  "excluir": ["cebollin", "morada", "perla", "polvo", "frita", "crispy"],
  "medida": "g",
  "factor": 1
}
```

El orden de los filtros no es casual. La categoría hace el trabajo pesado: dentro de `frutas-y-verduras/verduras` no hay mermelada de cebolla. Los términos eligen el ingrediente dentro de la categoría. Las exclusiones sacan la variedad que no corresponde: cebolla frita, betarraga cocida, zapallo en cubos.

> **POR QUÉ HAY UNA HERRAMIENTA SOLO PARA ESTO**
>
> Un mapeo malo **no falla**: cobra mal y nadie se entera. Una receta con 500 ml de leche pasaría de $650 a $4.370 sin que nada se rompa. `tools/verificar_fuentes.py` baja cada categoría, aplica el mapeo y compara el resultado con el precio de referencia del catálogo, ingrediente por ingrediente.

La verificación encontró, y se corrigieron: nueve fuentes apuntadas a una categoría equivocada, trece con la unidad mal declarada, cuatro con términos que no encontraban nada, y una docena con exclusiones que dejaban pasar la variedad cara. También fue la que destapó el asunto de la unidad «g» del capítulo 1.

Lo que no se pudo mapear quedó declarado

CADENA	FUENTES	ACTIVAS	INACTIVAS
Jumbo	98	96	2
Santa Isabel	98	87	11
Total	**196**	**183**	**13**

7.3 Resultados de la medición

2.608
productos considerados al verificar las 183 fuentes activas

+23 %
desvío mediano de Jumbo contra el precio de referencia escrito a mano

+18 %
el mismo desvío en Santa Isabel, sobre 1.088 productos

El rango del desvío va de **−65 % a +313 %** en Jumbo y de −60 % a +165 % en Santa Isabel. Esa dispersión es la misma conclusión del Hito 1, ahora sobre 2.608 productos en lugar de diecisiete: **no existe un factor por cadena**, y por eso el precio hay que medirlo.

Las dos corridas que dejaron los datos de la demostración

CADENA	CATEGORÍAS	PRODUCTOS LEÍDOS	PRECIOS ESCRITOS	DURACIÓN
Jumbo	52 / 52	3.085	96	6 min
Santa Isabel	52 / 52	1.849	87	5 min
Total	**104**	**4.934**	**183**	—

183 precios escritos sobre 183 fuentes activas: ninguna quedó sin medición. Ese volcado está en `db/003_snapshot_precios.sql`, de modo que una base recién creada ya trae precios medidos y la demostración no depende de que el sitio de la cadena responda en la sala.

CAPÍTULO 8

Cómo se verifica que esto funciona

Tres redes distintas, porque cada una atrapa cosas distintas.

1 · Pruebas automatizadas sobre la capa de negocio

`./gradlew -p server test` · 33 pruebas, sin base de datos y sin red, porque el normalizador y el parser son Kotlin puro. Corren en segundos, que es lo que hace que se corran.

QUÉ SE PRUEBA	LO QUE ATRAPA
Las conversiones de unidad	el error de factor diez de la unidad «g»
El percentil 25	que con pocos candidatos tire al barato, y que sin candidatos no devuelva cero: una receta saldría gratis
Términos, exclusiones y tildes	que «champinon» encuentre «Champiñón»
Los factores de conversión	el ajo por diente y la palta por unidad
El parser del catálogo	que una página sin catálogo devuelva vacío en lugar de reventar: es lo que va a pasar cuando la cadena cambie su sitio
La configuración	que el servicio no arranque sin clave, y que el error no imprima la clave

2 · Verificación del contrato de la interfaz

`bash tools/probar_api.sh` · 48 verificaciones contra el servicio corriendo, cada una con el código de estado que espera. Incluye los casos de error, que son los que se rompen en silencio: el 401 sin clave, el 409 del segundo `POST`, el 400 con los campos nombrados, el 404 después del `DELETE`.

La misma batería está en la colección de Postman, con 35 peticiones en tres carpetas: la demostración en orden, los errores y el resto del contrato. Cada petición lleva su prueba de estado, así que la colección se puede correr entera.

3 · Verificación del mapeo contra el sitio real

`python tools/verificar_fuentes.py --cadena JUMBO` · las 98 fuentes de una cadena, con el precio que daría cada una y su desvío contra la referencia. Es la única red que atrapa el error que no se puede atrapar de otra forma: la fuente que apunta a la categoría equivocada y devuelve un número perfectamente creíble.

> **LO QUE ESTAS TRES REDES NO CUBREN**
>
> No hay pruebas de integración contra una base de verdad: los DAO se ejercitan a través del servicio corriendo, no en aislamiento. Es la deuda más clara que este hito deja, y la primera que conviene pagar en el Hito 3 cuando el cliente empiece a escribir contra el servicio.

CONCLUSIÓN

Lo que queda construido y lo que queda debiendo

El Hito 1 prometió un precio real y dejó anotado lo que faltaba para tenerlo. Este hito lo construyó: hay un servicio con un contrato de 22 rutas, cuatro capas que se pueden verificar abriendo archivos, una base que se crea desde cero con un script, y un recolector que leyó 4.934 productos entre las dos cadenas y dejó 183 precios medidos, uno por cada fuente activa.

La medición amplió el argumento original. Donde el Hito 1 comparaba diecisiete productos, ahora hay 2.608, y la conclusión se sostiene mejor: el desvío contra los precios escritos a mano tiene una mediana de +23 % y un rango de −65 % a +313 %. No hay un factor por cadena. El precio no se deriva: se mide.

Tres cosas que este hito deja mejor de lo planificado

— La cadena publica el precio por unidad de medida, así que desapareció la pieza más frágil del diseño: deducir el tamaño del envase desde el nombre del producto.

— El precio dejó de ser un valor y pasó a ser una historia, con la corrida que lo produjo anotada al lado. Un precio raro ahora se puede rastrear.

— `origen` es una columna y no una etiqueta de pantalla. La app no puede prometer precisión de más por descuido.

Tres deudas declaradas

— **Trece fuentes inactivas.** Dos ingredientes no se venden en línea en ninguna de las dos cadenas y nueve más faltan en Santa Isabel. Su precio queda `ESTIMADO`, que es verdad, pero es cobertura que falta.

— **Sin pruebas de integración.** Los DAO se ejercitan a través del servicio, no en aislamiento.

— **El cliente todavía no consume el servicio.** La app sigue leyendo su JSON empaquetado. Conectarla es el trabajo del Hito 3, y la decisión del Hito 1 de que el dominio defina `PrecioRepository` como interfaz es lo que lo hace una implementación nueva y no una refactorización.

> LO QUE VIENE
>
> El Hito 3 conecta la app al servicio, muestra «medido» o «estimado» en la pantalla, y agrega la lista de compras de la semana. El contrato ya está cerrado y verificado, así que ese trabajo es de integración y no de descubrimiento.

BIBLIOGRAFÍA

Referencias

Norma APA, séptima edición. Cada una sostiene una decisión concreta del documento; la decisión que sostiene va anotada en cursiva.

Docker Inc. (2026). *Compose file reference*. https://docs.docker.com/reference/compose-file/
Capítulo 4.3: orden de ejecución de los scripts de inicialización y verificación de salud.

Fielding, R., Nottingham, M., & Reschke, J. (2022). *HTTP semantics* (RFC 9110). Internet Engineering Task Force. https://doi.org/10.17487/RFC9110
Capítulo 2.2: significado de 201, 202, 204, 409 y 503, y de la cabecera Location.

JetBrains. (2026). *Kotlin documentation*. https://kotlinlang.org/docs/home.html
Capítulo 5.1: nulabilidad en el sistema de tipos y corrutinas.

JetBrains. (2026). *Ktor documentation: Creating server plugins*. https://ktor.io/docs/server-custom-plugins.html
Capítulo 2.3: el plugin que verifica la clave en cada llamada.

JetBrains. (2026). *Ktor documentation: Status pages*. https://ktor.io/docs/server-status-pages.html
Capítulo 2.2: traducción de excepción a código de estado en un solo lugar.

Oracle. (2026). *Java Platform, Standard Edition 21 API specification: java.security.MessageDigest*. https://docs.oracle.com/en/java/javase/21/docs/api/
Capítulo 2.3: comparación de tiempo constante de la clave.

PostgreSQL Global Development Group. (2026). *PostgreSQL 18 documentation*. https://www.postgresql.org/docs/18/
Capítulo 4.2: DISTINCT ON, arreglos de texto, timestamptz y numeric para dinero.

Wooldridge, B. (2026). *HikariCP: Down the rabbit hole*. https://github.com/brettwooldridge/HikariCP
Capítulo 3.2: dimensionamiento del pozo de conexiones.

Instituto Nacional de Estadísticas. (2023). *VIII Encuesta de Presupuestos Familiares*. INE Chile.
Capítulo de introducción: el gasto del hogar chileno en alimentos, citado desde el Hito 1.

Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2.ª ed.). O'Reilly Media.
Capítulo 6: criterios para no dividir todavía, y dónde dejar las costuras.

SOBRE LAS FUENTES DE LOS DATOS MEDIDOS

Las cifras de precios de este documento no vienen de ninguna publicación: son mediciones propias hechas contra *jumbo.cl* y *santaisabel.cl* el 29 de octubre de 2026 con las herramientas del repositorio, y son reproducibles ejecutándolas.

ANEXO A

Tabla completa de rutas

El contrato completo, con los cuerpos de ejemplo y las reglas de cada campo, está en `docs/api-cookify.md` y en el repositorio.

#	VERBO	RUTA	CLAVE	ÉXITO	ERRORES
1	GET	`/salud`	no	200	503
2	GET	`/api/v1/supermercados`	no	200	—
3	GET	`/api/v1/supermercados/{codigo}`	no	200	404
4	GET	`/api/v1/ingredientes`	no	200	400
5	GET	`/api/v1/ingredientes/{id}`	no	200	404
6	POST	`/api/v1/ingredientes`	sí	201	400 · 401 · 409
7	PUT	`/api/v1/ingredientes/{id}`	sí	200	400 · 401 · 404
8	DELETE	`/api/v1/ingredientes/{id}`	sí	204	401 · 404
9	GET	`/api/v1/fuentes`	no	200	400
10	GET	`/api/v1/fuentes/{id}`	no	200	400 · 404
11	POST	`/api/v1/fuentes`	sí	201	400 · 401 · 404 · 409
12	PUT	`/api/v1/fuentes/{id}`	sí	200	400 · 401 · 404
13	DELETE	`/api/v1/fuentes/{id}`	sí	204	401 · 404
14	GET	`/api/v1/precios`	no	200	400 · 404
15	GET	`/api/v1/precios/{id}`	no	200	400 · 404
16	POST	`/api/v1/precios`	sí	201	400 · 401 · 404 · 409
17	PUT	`/api/v1/precios/{id}`	sí	200	400 · 401 · 404
18	DELETE	`/api/v1/precios/{id}`	sí	204	401 · 404
19	GET	`/api/v1/recolecciones`	no	200	400
20	GET	`/api/v1/recolecciones/{id}`	no	200	400 · 404
21	POST	`/api/v1/recolecciones`	sí	202	400 · 401 · 404 · 409 · 503
22	DELETE	`/api/v1/recolecciones/{id}`	sí	204	401 · 404 · 409

Parámetros de consulta

RUTA	PARÁMETROS
`/ingredientes`	`pasillo` · `q` · `pagina` · `tamano`
`/fuentes`	`supermercado` · `ingredienteId` · `activa` · `pagina` · `tamano`
`/precios`	`supermercado` · `origen` · `ingredienteId` · `historia` · `pagina` · `tamano`
`/recolecciones`	`supermercado` · `pagina` · `tamano`

ANEXO B

Salida de las verificaciones

Extractos reales, reproducibles ejecutando los comandos que los encabezan.

Las 48 verificaciones del contrato

```
$ set -a && . ./.env && set +a && bash tools/probar_api.sh

== ingredientes: las cuatro operaciones ==
  ok   200  GET /ingredientes
  ok   400  GET con pasillo inexistente
  ok   404  GET de uno que no existe
  ok   401  POST sin clave
  ok   401  POST con clave equivocada
  ok   201  POST crea el ingrediente
  ok   409  POST repetido no reemplaza en silencio
  ok   400  POST con precio negativo y unidad invalida
  ok   200  PUT actualiza
  ok   204  DELETE borra
  ok   404  GET despues del DELETE
...
ok: 48   fallos: 0
```

Las 33 pruebas de la capa de negocio

```
$ ./gradlew -p server test

NormalizadorTest > un precio en gramos viene por cada diez gramos, no por gramo() PASSED
NormalizadorTest > el ajo se cobra por diente aunque la cadena venda la cabeza() PASSED
NormalizadorTest > la palta se cobra por unidad aunque la cadena la venda por kilo() PASSED
NormalizadorTest > sin candidatos no hay precio, y eso no es cero() PASSED
ParserCatalogoTest > una pagina sin catalogo devuelve lista vacia y no revienta() PASSED
ConfiguracionTest > una clave de api demasiado corta no se acepta() PASSED
...
BUILD SUCCESSFUL
```

La verificación del mapeo contra el sitio

```
$ python tools/verificar_fuentes.py --cadena JUMBO

ingrediente               estado        medido referencia  desvio  cand.  producto p25
pechuga_pollo             ok             9.182       8.90     +3%     28  Pechuga Deshuesada
de Pollo 850 g
leche                     ok             1.410       1.30     +8%    174  Leche Surlat
Descremada 1 L
lentejas                  ok             3.853       3.20    +20%     20  Lentejas 6 mm
Cuisine & Co 1 kg
oregano                   ok            23.000      30.00    -23%      9  Orégano Entero
Gourmet 50 g
ajo                       ok            47.100      60.00    -21%      3  Ajo 3 un.
palta                     ok           738.000     900.00    -18%      4  Palta Hass Malla 1
kg
tocino                    DESVIO        23.189      11.00   +111%     11  Tocino Cortado La
Preferida 180 g
tofu                      inactiva           -       8.00
...
98 fuentes, 25 con problema
```

Los 25 «con problema» de Jumbo son todos desvíos contra el precio de referencia, no errores de mapeo: ninguna fuente quedó sin candidatos. Ese desvío **es** el hallazgo del proyecto, no un defecto de la herramienta.